MEDROUTE

Emergency Smart Ambulance Routing & Hospital Resource Dispatcher

Complete Build Plan – Pune Edition

MCA Project | Java + Spring Boot + PostgreSQL/PostGIS + React

Scope note: MedRoute is an operational decision-support and coordination platform. All patient vitals, traffic and hospital resource numbers are **simulated**. It does not diagnose patients.

1. Project Snapshot

MedRoute connects an ambulance, its patient telemetry, live road conditions and hospital resources in one system, so the ambulance is sent to a hospital that can actually treat the patient quickly, not just the nearest one. This plan turns your project document into an ordered build with clear deliverables.

Item	Decision
City / map area	Pune, Maharashtra. Center about 18.5204, 73.8567.
Road data bounding box	West 73.75, South 18.40, East 73.98, North 18.65 (trim later if the file is too large).
Core algorithms	A* with haversine heuristic (main), Dijkstra (reference), dynamic edge weights.
Backend	Java 21, Spring Boot 3.x, Spring Security + JWT, Spring WebSocket (STOMP).
Database	PostgreSQL 16 with PostGIS and pgRouting (via Docker).
Frontend	React (Vite), Leaflet + OpenStreetMap tiles, SockJS/STOMP client.
Data	Real Pune roads (OSM) + real hospital names/locations; simulated resources, vitals and traffic.
Testing / build	JUnit 5, Mockito, Maven, Git + GitHub.

Roles and what each one sees

Role	Main screens	Key actions
ADMIN	Admin dashboard, reports	Manage users, hospitals, ambulances; view emergency history; tune scoring weights.
AMBULANCE	Ambulance dashboard	Start emergency, enter patient data, see recommended hospital, route, ETA, live status.
HOSPITAL	Hospital ER dashboard	Update resources, see incoming patients with live vitals and ETA, acknowledge alerts.

2. Prerequisites and Tools

Tool	Version	Purpose
JDK	21 (LTS)	Backend language.
Maven	3.9+	Build and dependency management.
Node.js	20 LTS+	React frontend.
Docker Desktop	Latest	Run PostgreSQL + PostGIS + pgRouting without manual installs.
IDE	IntelliJ IDEA / VS Code	Development.
Postman	Latest	API testing.
osm2pgrouting	Latest	Imports OSM road data into pgRouting tables.
Git + GitHub	Latest	Version control; commit after every phase.

Spring Boot dependencies (start.spring.io)

- Spring Web, Spring Security, Spring Data JPA, Validation, WebSocket
- PostgreSQL Driver, Lombok (optional), Spring Boot DevTools
- JJWT (io.jsonwebtoken) for JWT; Hibernate Spatial if you map geometry columns
- Test: Spring Boot Starter Test (JUnit 5, Mockito), Spring Security Test

Docker Compose for the database

```
services:
  db:
    image: pgrouting/pgrouting:latest     # check Docker Hub for a pinned tag
    environment:
      POSTGRES_DB: medroute
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
volumes:
  pgdata:
```

Then run: `CREATE EXTENSION postgis; CREATE EXTENSION pgrouting;` in the medroute database.

3. 10-Week Timeline

Compress or stretch to your deadline. The order matters more than the exact weeks: routing and selection are the heart of the project, so finish them before polishing the UI.

Week	Focus	Output
1	Setup, database, repo	Running Spring Boot app connected to PostGIS; React skeleton; GitHub repo.
2	Authentication and CRUD	JWT login with 3 roles; APIs for ambulances, hospitals, resources, patients.
3	Pune road network	OSM data imported; graph visible; hospitals seeded with real coordinates.
4	Graph in memory + Dijkstra	Adjacency list loaded from DB; Dijkstra returns correct Pune routes.
5	A*, dynamic weights	A* with haversine; traffic multipliers; A* vs Dijkstra comparison table.
6	Simulators (multithreading)	Traffic, GPS movement and heart rate/SpO2 threads running safely.
7	Hospital selection + emergency flow	Scoring engine; full emergency lifecycle via REST.
8	WebSocket + dashboards	Live telemetry, ETA, alerts pushed to ambulance and hospital screens.
9	Maps, reports, polish	Leaflet route display; admin reports; validation and error handling.
10	Testing, docs, demo	JUnit suite, final report, demo script, presentation.

4. Phase-by-Phase Build Guide

Phase 1: Foundation (Week 1)

Goal: A clean project skeleton that runs end to end.

- Create GitHub repo with folders **backend/**, **frontend/**, **docs/**, **data/**.
- Generate the Spring Boot project; add the Docker Compose file; connect via application.yml.
- Create the React app with Vite; add a proxy to the backend (port 8080).
- Add a health endpoint `GET /api/health` and call it from React.

Deliverable: Backend, DB and frontend all start and talk to each other.

Done when: React page shows the health message returned by Spring Boot.

Phase 2: Authentication and Core CRUD (Week 2)

Goal: Secure login and the basic data entities.

- Entities and repositories: users, ambulances, patients, hospitals, hospital_resources.
- Spring Security: BCrypt password hashing, JWT filter, role-based route protection.
- Endpoints: `/api/auth/login`, then CRUD per entity, restricted by role.
- Use DTOs and validation annotations; add a global exception handler.
- Seed an admin user, 3 ambulance users and 5 hospital users on startup.

Deliverable: Postman collection covering login and every CRUD endpoint.

Done when: Each role can log in and is blocked from other roles' endpoints (403).

Phase 3: Pune Road Network and Data (Week 3)

Goal: Real Pune roads inside PostgreSQL, plus seeded hospitals.

- Download roads for the Pune bounding box from OpenStreetMap (see Section 5).
- Import with osm2pgrouting; confirm tables **ways** and **ways_vertices_pgr** exist.
- Select 8-10 Pune hospitals; look up exact coordinates on OpenStreetMap and store in hospitals.
- Snap each hospital to its nearest graph vertex and store the vertex id (node_id).
- Create 3-4 ambulance start points (stations) and snap them to vertices the same way.
- Run one pgRouting query (pgr_dijkstra) between two vertices as a sanity check.

Deliverable: Database containing the Pune road graph, hospitals with node_id, ambulances with node_id.

Done when: A pgr_dijkstra query returns a believable route between two Pune hospitals.

Phase 4: Graph in Memory and Dijkstra (Week 4)

Goal: Your own DSA implementation, independent of pgRouting.

- Load edges from the **ways** table at startup into an adjacency list: Map<Long, List<Edge>>.
- Edge fields: edgeId, from, to, lengthMeters, baseSpeed, trafficMultiplier (default 1.0).
- Node fields: id, latitude, longitude (needed for the A* heuristic and map drawing).
- Implement Dijkstra with a PriorityQueue; return path (node list), total cost, nodes visited.
- Compare your result to pgr_dijkstra for 10 random pairs; they should match.

Deliverable: RoutingService with `dijkstra(source, target)` plus a test class.

Done when: Your path cost equals pgRouting's cost on all test pairs.

Phase 5: A* and Dynamic Traffic Weights (Week 5)

Goal: The main algorithm with live-changing road costs.

- Implement A* with f(n) = g(n) + h(n); h(n) = haversine distance to target divided by max speed.
- Edge cost = (length / speed) x trafficMultiplier. Blocked road = Double.MAX_VALUE (skip it).

- Keep the heuristic **admissible** (never overestimate), otherwise A* is not guaranteed optimal.
- Build a comparison endpoint returning nodes visited and time taken for A* and Dijkstra.
- Add a reroute method that recomputes from the ambulance's current node when traffic changes.

Deliverable: RoutingService.aStar(...) and a benchmark table for 20 Pune origin-destination pairs.

Done when: A* gives the same cost as Dijkstra but visits fewer nodes; blocking a road changes the route.

Phase 6: Simulators and Multithreading (Week 6)

Goal: Realistic live data without real sensors.

- **TrafficSimulator:** ScheduledExecutorService, every 5-10 s changes multipliers on selected Pune corridors (normal 1.0, moderate 1.6, heavy 2.5, blocked).
- **GpsSimulator:** one task per active ambulance moves it node by node along its route.
- **VitalsSimulator:** one task per patient generates heart rate and SpO_2 with small random drift; occasionally triggers a warning value.
- Use ConcurrentHashMap for edge multipliers and AtomicReference/volatile for shared state; never share a plain HashMap between threads.
- Provide start/stop controls so tests and demos are repeatable.

Deliverable: Simulation package with thread-safe shared state.

Done when: Logs show independent threads updating without exceptions for 10+ minutes.

Phase 7: Hospital Selection and Emergency Workflow (Week 7)

Goal: The decision engine and the full lifecycle of one emergency.

- Implement filtering and scoring (Section 8).
- Emergency lifecycle states: CREATED, AMBULANCE_ASSIGNED, PATIENT_PICKED, HOSPITAL_SELECTED, EN_ROUTE, ARRIVED, CLOSED.
- Endpoints for create emergency, assign nearest free ambulance, submit patient data, select hospital, start trip, close case.
- Re-evaluate hospital choice when resources or traffic change materially (with a cooldown so it does not flip every second).
- Reproduce your document's example (Hospital A no ICU, B has ICU + blood, C heavy traffic) as a test.

Deliverable: EmergencyService and HospitalSelectionService with unit tests.

Done when: The example scenario picks Hospital B in the automated test.

Phase 8: Real-Time Layer and Dashboards (Week 8)

Goal: Updates appear without refreshing.

- Configure STOMP over WebSocket with SockJS fallback; authenticate the socket using the JWT.
- Publish telemetry, ETA, route changes and alerts to the topics in Section 10.
- React: login page, role-based routing, ambulance dashboard, hospital dashboard, admin dashboard.
- Hospital dashboard shows incoming patient cards (vitals, blood group, trauma level, ETA) and updates live; ambulance dashboard shows the recommended hospital and countdown ETA.

Deliverable: Working live screens for all three roles.

Done when: Changing hospital resources or vitals updates the other screen within about 1-2 seconds.

Phase 9: Maps, Reports and Polish (Week 9)

Goal: Visual impact and completeness.

- Leaflet map centered on Pune: hospital markers, moving ambulance marker, route polyline.
- Traffic overlay: color route segments by multiplier (green / orange / red).
- Admin reports: emergency history table, average ETA, hospital usage, filter by date.

- Input validation, loading and error states, consistent API error format, logging.

Deliverable: Polished UI with map and reports.

Done when: A full emergency can be demonstrated visually from request to closure.

Phase 10: Testing, Documentation and Demo (Week 10)

Goal: Proof that it works and a clean submission.

- Run the test plan (Section 11); fix defects; aim for meaningful coverage of routing and selection.
- Write the report: problem, design, algorithms, results table (A* vs Dijkstra), screenshots, limitations.
- Record a short demo video as a backup in case the live demo fails.
- Tag the final GitHub release and write a README with run instructions.

Deliverable: Final report, README, demo video, presentation slides.

Done when: A new machine can run the project by following the README only.

5. Pune Data Setup

Step 1: Get the road data

Option A: Overpass API (small areas). Option B: a custom area extract from a service such as BBBike or a Geofabrik extract clipped to Pune with osmium. If the file is too large, shrink the bounding box.

```
# Overpass bounding box order: west,south,east,north
curl -o data/pune.osm "https://overpass-api.de/api/map?bbox=73.75,18.40,73.98,18.65"
```

Step 2: Import into pgRouting

```
osm2pgrouting --f data/pune.osm --conf mapconfig.xml \
              --dbname medroute --username postgres --clean
```

The default **mapconfig.xml** that ships with osm2pgrouting is enough: it keeps drivable roads. After import you get **ways** (edges with length and cost) and **ways_vertices_pgr** (nodes).

Step 3: Sanity-check the graph

```
SELECT count(*) FROM ways;  SELECT count(*) FROM ways_vertices_pgr;

-- nearest graph vertex to a point (longitude, latitude)
SELECT id FROM ways_vertices_pgr
ORDER BY the_geom <-> ST_SetSRID(ST_MakePoint(73.8567, 18.5204), 4326) LIMIT 1;

SELECT * FROM pgr_dijkstra(
  'SELECT gid AS id, source, target, cost, reverse_cost FROM ways',
  <source_vertex>, <target_vertex>, true);
```

Hospitals to seed (look up exact coordinates on OSM)

Suggested list: Sassoon General Hospital, Ruby Hall Clinic, Jehangir Hospital, Poona Hospital, Deenanath Mangeshkar Hospital, KEM Hospital, Sahyadri Hospital, Aditya Birla Memorial Hospital. Use real names and locations for realism, but the ICU bed, ventilator and blood counts are invented simulation values. Say so in your report.

6. Database Design

Start with the tables in your project document, then add the few fields the routing engine needs.

Table	Key fields (document) + additions marked NEW
users	user_id, name, email, password_hash, role
ambulances	ambulance_id, vehicle_number, driver_id, latitude, longitude, status; NEW: node_id, geom
patients	patient_id, ambulance_id, age, blood_group, trauma_level, heart_rate, spo2
hospitals	hospital_id, name, latitude, longitude, status; NEW: node_id, geom
hospital_resources	hospital_id, icu_available, emergency_beds, ventilators, blood_units; NEW: updated_at (optional: hospital_blood_stock by blood group)
emergency_requests	request_id, ambulance_id, patient_id, hospital_id, severity, status, created_at; NEW: closed_at, eta_seconds
routes	route_id, source, destination, distance, traffic_weight, estimated_time; NEW: path_json
NEW: telemetry_log	id, patient_id, heart_rate, spo2, recorded_at (history for reports)
NEW: traffic_events	id, edge_id, multiplier, started_at, ended_at (explain reroutes in the demo)

Tip: blood availability is more realistic per blood group. The single blood_units field works for a first version; the optional hospital_blood_stock table (hospital_id, blood_group, units) lets selection match the patient's group exactly.

7. Project Structure

```
medroute/
  docker-compose.yml
  data/                    pune.osm, seed SQL
  docs/                    report, diagrams, screenshots
  backend/
    pom.xml
    src/main/java/com/medroute/
      auth/                User, JwtService, SecurityConfig, AuthController
      ambulance/           Ambulance entity, repo, service, controller
      patient/             Patient entity, repo, service, controller
      hospital/            Hospital, HospitalResource, repos, services, controllers
      routing/             Graph, Node, Edge, AStarRouter, DijkstraRouter, RoutingService
      selection/           HospitalSelectionService, ScoringConfig
      emergency/           EmergencyRequest, lifecycle service, controller
      simulation/          TrafficSimulator, GpsSimulator, VitalsSimulator
      websocket/           WebSocketConfig, TelemetryPublisher
      common/              DTOs, exceptions, utils (Haversine)
    src/test/java/...      routing, selection, auth, emergency tests
  frontend/
    src/pages/             Login, AmbulanceDashboard, HospitalDashboard, AdminDashboard
    src/components/        MapView, PatientCard, ResourcePanel, AlertList
    src/services/          api.js, socket.js
```

8. Core Logic Design

8.1 Graph model

- **Node** = road intersection (vertex id, lat, lon). **Edge** = road segment (length in meters, base speed).
- Cost in seconds = (length / speed) x trafficMultiplier. Using time (not distance) makes ETA a direct output.
- Multipliers: normal 1.0, moderate 1.6, heavy 2.5, blocked = edge ignored.

8.2 A* skeleton (Java)

```
double[] g = new double[maxNodeId + 1];  Arrays.fill(g, Double.MAX_VALUE);
Map<Long, Long> cameFrom = new HashMap<>();
PriorityQueue<double[]> open =                 // {f, nodeId}
        new PriorityQueue<>((a, b) -> Double.compare(a[0], b[0]));
g[start] = 0;
open.add(new double[]{ heuristic(start, goal), start });

while (!open.isEmpty()) {
    double[] cur = open.poll();
    long u = (long) cur[1];
    if (u == goal) return buildPath(cameFrom, goal, g[goal]);
    if (cur[0] > g[(int) u] + heuristic(u, goal) + 1e-9) continue;   // stale entry
    for (Edge e : graph.edgesFrom(u)) {
        if (e.isBlocked()) continue;
        double cost = g[(int) u] + e.lengthMeters() / e.speed() * e.trafficMultiplier();
        if (cost < g[(int) e.to()]) {
            g[(int) e.to()] = cost;
            cameFrom.put(e.to(), u);
            open.add(new double[]{ cost + heuristic(e.to(), goal), e.to() });
        }
    }
}
return noPathFound();

// heuristic = haversine(node, goal) / maxSpeedMetersPerSecond   (never overestimates)
```

Dijkstra is the same loop with heuristic = 0. Keeping both in one class makes your comparison fair.

8.3 Hospital selection

Step 1 - hard filters (a hospital failing any is rejected): status ACTIVE; at least 1 ICU bed if trauma level is critical; at least 1 emergency bed; ventilator available if SpO_2 is below a threshold (for example 90); blood available for the patient's group.

Step 2 - score the remaining hospitals (lower is better), each term normalized to 0-1 across candidates:

```
score = wEta * etaNorm + wTraffic * trafficNorm + wResource * (1 - resourceMatch)
default weights:  wEta = 0.5,  wTraffic = 0.2,  wResource = 0.3   (stored in config / admin screen)
```

- **eta**: A* travel time from the ambulance's current node to the hospital node.
- **traffic**: average multiplier along the computed route.
- **resourceMatch**: how comfortably the hospital covers the patient's needs (spare ICU beds, ventilators, blood).
- If every hospital is filtered out, relax the least critical filter and raise a visible alert.

8.4 Why this meets your example scenario

Hospital	Situation	Result
A	Closest, no ICU bed	Rejected by hard filter.
C	Nearby, route in heavy traffic	ETA grows (multiplier 2.5), score worsens.
B	Slightly farther, ICU + blood available	Best total score; selected.

9. REST API Plan

Method	Endpoint	Role	Purpose
POST	/api/auth/login	All	Returns JWT and role.
GET/POST/PUT	/api/ambulances	ADMIN, AMBULANCE	Manage ambulances; update status and position.
GET/POST	/api/hospitals	ADMIN, HOSPITAL	Hospital list and details.
PUT	/api/hospitals/{id}/resources	HOSPITAL, ADMIN	Update ICU, beds, ventilators, blood.
POST	/api/emergencies	AMBULANCE	Create emergency; assigns ambulance.
PUT	/api/emergencies/{id}/patient	AMBULANCE	Submit patient profile and vitals.
POST	/api/emergencies/{id}/select-hospital	AMBULANCE	Run selection; returns ranked hospitals.
POST	/api/emergencies/{id}/start	AMBULANCE	Compute route and begin trip simulation.
POST	/api/emergencies/{id}/close	AMBULANCE, HOSPITAL	Mark arrival and close case.
GET	/api/routes/compare?from=&to=	ADMIN	A* vs Dijkstra metrics.
POST	/api/simulation/traffic/block	ADMIN	Block or slow a road (demo control).
GET	/api/reports/emergencies	ADMIN	History with filters.

10. WebSocket Design

STOMP endpoint: `/ws`. Clients send the JWT when connecting; the server checks role before allowing a subscription.

Topic	Subscriber	Payload
/topic/ambulance/{id}/status	Ambulance dashboard	Position, speed, state, remaining distance.
/topic/ambulance/{id}/route	Ambulance dashboard	Updated path and ETA after a reroute.
/topic/hospital/{id}/incoming	Hospital dashboard	Patient summary, selected hospital, ETA.
/topic/patient/{id}/vitals	Hospital dashboard	Heart rate, SpO2, timestamp.
/topic/alerts	All (filtered)	Low SpO2, road blocked, hospital resource change, reroute.

- Publish only on **meaningful change** (new position, vital out of range, ETA changed by more than a few seconds) to avoid flooding the dashboards.
- Message envelope: `{ type, emergencyId, timestamp, data }` so the frontend can route messages with one switch statement.

11. Testing Plan

Area	Test cases
Routing	Known small graph (hand-computed answer); A* cost equals Dijkstra cost; blocked edge forces detour; no-path case; A* visits fewer nodes on Pune pairs.
Selection	Example scenario picks B; hospital without ICU rejected for critical patient; all hospitals rejected path; weight change flips the winner.
Security	Wrong password rejected; AMBULANCE cannot call admin APIs; expired or tampered JWT returns 401.
Emergency flow	Lifecycle states advance in order; closing frees the ambulance; invalid transition rejected.
Concurrency	Traffic updates while routes are being computed cause no exceptions; no lost updates to shared multiplier map.
WebSocket	Hospital topic receives vitals update after the simulator emits one; unauthorized subscription refused.
End to end	Scripted run: create emergency, select hospital, trip, block road, reroute, arrive, close.

12. Demo Script (about 7 minutes)

- **1.** Log in as Hospital staff in one browser window and Ambulance staff in another.
- **2.** Ambulance: create an emergency at a Pune location, enter a critical patient (blood group, low SpO_2).
- **3.** Show ranked hospitals and the reason the nearest one was rejected (no ICU bed).
- **4.** Start the trip: route on the map, ETA counting down, vitals streaming to the hospital dashboard.
- **5.** Admin: block a road on the route; the route recalculates and both dashboards update.
- **6.** Hospital: reduce ICU beds to 0 mid-trip; selection re-evaluates and alerts the ambulance.
- **7.** Arrival and closure; open the admin report showing this case and the A* vs Dijkstra table.

Likely viva questions to prepare

- Why A* over Dijkstra? What makes the heuristic admissible?
- How do dynamic weights change the graph without rebuilding it?
- How do you keep shared data thread-safe across simulators?
- Why STOMP/WebSocket instead of polling? What if the connection drops?
- Why is hospital selection not just nearest-first? How are weights chosen and tuned?
- How is the system secured and how are roles enforced?
- What are the limitations and how would live traffic APIs and predictive ETA fit in?

13. Risks and Tips

Risk	Mitigation
OSM file too big or import slow	Shrink the bounding box (central Pune plus main corridors); keep only drivable highway types.
Graph too large for memory	Pune extract is usually manageable; if not, prune minor service roads.
Heuristic overestimates (wrong routes)	Divide haversine by the maximum speed in the graph.
Thread bugs in simulators	Use concurrent collections, avoid shared mutable objects, keep one writer per value.
WebSocket flooding	Throttle publishing; send diffs, not full state.
Demo failure	Seeded scenario button, pre-recorded video, fixed random seed for simulators.
Scope creep	Do not start Future Enhancements (live traffic API, mobile app, cloud) until all 10 phases pass.

Working habits

- Commit at the end of every phase and tag it (v0.1 ... v1.0).
- Build the smallest working version of each module first, then improve.
- Keep a changelog in docs/ - it becomes the 'implementation' chapter of your report.
- Label all simulated data clearly in the UI and the report.

14. Final Submission Checklist

- Source code on GitHub with README and run steps
- Database scripts and Pune seed data
- Postman collection and API list
- Architecture diagram, ER diagram, flow diagram, algorithm explanation
- A* vs Dijkstra results table with Pune route examples
- JUnit test report and screenshots of every dashboard
- Demo video and presentation slides
- Limitations and future enhancements section